Documento

Manual de Instalación



Escuela: Ing. E. Latzina.

Materia: Prácticas Profesionalizantes.

Docente: Serrano Aaron, Moya Juan.

Alumnos: Domínguez Victoria, Galíndez Sofía, Goncalves Luna, Iannopollo Angeline.

Curso: 6°2°, T.M, C.

Ciclo Lectivo: 2026.

Tabla de Contenidos

Sistema de Gestión de Notas..... 3

1. Introducción..... 3

Objetivo..... 3

Alcance..... 3

Público destinatario..... 3

Descripción breve del sistema..... 3

2. Arquitectura general..... 3

3. Requisitos previos..... 4

4. Descarga / preparación del proyecto..... 5

5. Configuración de PostgreSQL..... 6

6. Configuración del Backend..... 6

7. Configuración del correo electrónico..... 7

8. Configuración del Frontend..... 8

9. Ejecución del sistema..... 8

10. Verificación de la instalación..... 8

11. Inicialización de datos (Sembrado / Seed)..... 9

12. Configuración para desarrollo..... 9

13. Configuración para un entorno de producción..... 9

14. Copias de seguridad de PostgreSQL..... 9

15. Solución de problema frecuente..... 9

16. Detención del sistema..... 10

17. Estructura de carpetas representativa..... 10

Sistema de Gestión de Notas

1. Introducción

Objetivo

Este manual describe el procedimiento detallado para instalar, configurar, ejecutar y verificar desde cero el Sistema de Gestión de Notas (SGN) de la Escuela Ing. E. Latzina (ET35) en una computadora con sistema operativo Microsoft Windows o Linux.

Alcance

El documento abarca el aprovisionamiento del entorno, la preparación de la base de datos PostgreSQL, la compilación e inicio del servicio backend (Spring Boot) y la ejecución del servicio frontend (Next.js 14).

Público destinatario

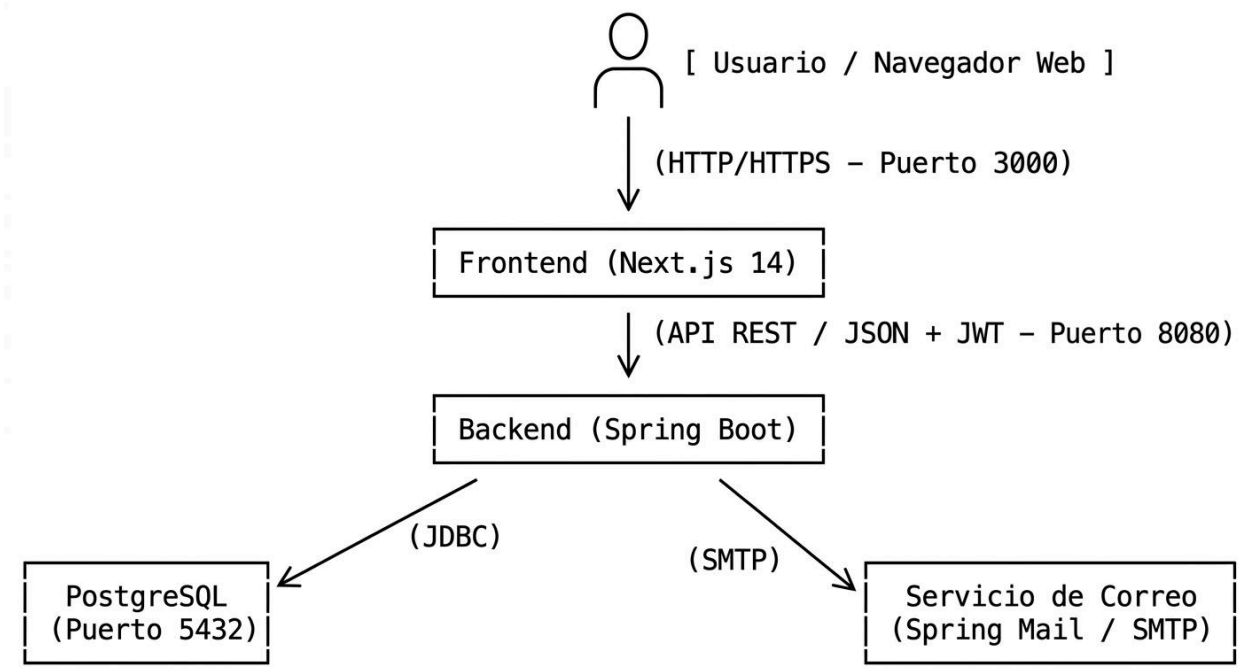
Dirigido a administradores de sistemas, personal técnico o encargados del despliegue del sistema. Se asumen conocimientos básicos de manejo de terminal de comandos y administración de servicios web.

Descripción breve del sistema

El SGN es una plataforma web monolítica dividida en dos componentes principales: un frontend interactivo y receptivo y un backend de servicios REST. Permite la administración integral de cursos, materias, calificaciones, boletines e inasistencias de la institución.

2. Arquitectura general

El sistema sigue un modelo desacoplado basado en arquitectura Cliente-Servidor mediante API REST:



- Frontend: Desarrollado en Next.js 14 con React y TypeScript. Se ejecuta en el puerto predeterminado 3000.
- Backend: API REST desarrollada en Java 17 / 21 con Spring Boot. Se ejecuta en el puerto predeterminado 8080.
- Base de Datos: PostgreSQL para persistencia relacional en el puerto predeterminado 5432.
- Autenticación: Intercambio de tokens JWT en el encabezado Authorization: Bearer <token>.
- Servicio de Correo: Spring Mail integrado en el backend para envío de credenciales temporales y restablecimiento de contraseñas.

3. Requisitos previos

Para la ejecución del SGN en una máquina limpia, deben instalarse las siguientes dependencias:

Componente	**Función en el Proyecto**	**Versión Recomendada**	**Comando de Verificación**

Sistema Operativo	Entorno de ejecución	Windows 10/11 o Ubuntu 22.04 LTS	ver (Win) / uname -a (Linux)
Java Development Kit (JDK)	Compilación y ejecución del backend	Java 17 o Java 21	java -version
Apache Maven	Gestor de dependencias y build del backend	Maven 3.8+	mvn -version
Node.js	Entorno de ejecución para el frontend	Node.js LTS v18.x ó v20.x	node -v
npm	Gestor de paquetes JavaScript	npm 9.x ó 10.x	npm -v
PostgreSQL	Motor de base de datos relacional	PostgreSQL 14, 15 o 16	psql --version

4. Descarga / preparación del proyecto

El proyecto está estructurado como un repositorio unificado que contiene las carpetas del servidor y del cliente:

sgn-notas/
├── backend/
│ ├── src/
│ ├── pom.xml
│ └── application.yml

```
└── frontend/
    ├── app/
    ├── components/
    ├── lib/
    ├── package.json
    └── .env.local
```

5. Configuración de PostgreSQL

El backend requiere una instancia activa de PostgreSQL. Acceda a la consola interactiva de PostgreSQL mediante psql o una interfaz gráfica como pgAdmin: CREATE DATABASE sgn_db; CREATE USER sgn_user WITH ENCRYPTED PASSWORD 'TU_PASSWORD_POSTGRES'; GRANT ALL PRIVILEGES ON DATABASE sgn_db TO sgn_user;

El backend está configurado mediante Spring Data JPA / Hibernate. En el entorno de desarrollo, el parámetro hibernate.ddl-auto está establecido en update o create-drop en el archivo application.yml, por lo que Hibernate creará automáticamente todas las tablas e índices relacionales al iniciar la aplicación por primera vez. Nota: No se requiere la ejecución manual de scripts DDL iniciales salvo que se proporcione un script de datos semilla en la carpeta del proyecto.

6. Configuración del Backend

Para la configuración de variables en application.yml abra el archivo backend/src/main/resources/application.yml.Ajuste los valores o configure las variables de entorno correspondientes:

Variable	Obligatoria	Función	Valor de Ejemplo
SERVER_PORT	No	Puerto de escucha del backend	8080

SPRING_DATASOURCE_URL	Sí	Cadena de conexión JDBC	jdbc:postgresql://localhost:5432/sgn_db
SPRING_DATASOURCE_USERNAME	Sí	Usuario de la base de datos	sgn_user
SPRING_DATASOURCE_PASSWORD	Sí	Contraseña de la base de datos	TU_PASSWORD_POSTGRES
SPRING_MAIL_USERNAME	Sí	Remitente para envíos SMTP	maildela35@gmail.com
SPRING_MAIL_PASSWORD	Sí	Clave de aplicación/ SMTP	TU_APP_PASSWORD
JWT_SECRET	Sí	Clave para firmado de tokens	TU_CLAVE_SECRETA_FIRMA

7. Configuración del correo electrónico

El servicio Spring Mail integrado gestiona la entrega de contraseñas temporales al dar de alta usuarios o ejecutar restablecimientos. Utilice el servidor SMTP smtp.gmail.com en el puerto 587 con TLS habilitado (starttls.enable=true). En caso de usar Gmail,

genere una Contraseña de Aplicación desde la configuración de seguridad de la cuenta de Google institucional (requiere autenticación de dos pasos activa). Reemplace SPRING_MAIL_PASSWORD por la contraseña de aplicación de 16 caracteres. Si el servidor SMTP no se encuentra disponible o las credenciales son inválidas, el backend arrojará una excepción en la traza de logs durante la creación o reseteo de usuarios. Sin embargo, la transacción en base de datos se completa y la contraseña temporal es generada en el sistema.

8. Configuración del Frontend

Abra una terminal en la carpeta frontend/ (cd frontend) y ejecute la instalación de paquetes (npm install). Cree un archivo llamado .env.local en la raíz del directorio frontend/ con lo siguiente: NEXT_PUBLIC_API_URL=http://localhost:8080/api. Esta variable define el punto de enlace que utilizan las funciones de consumo API (ubicadas en lib/api.ts o componentes de React) para comunicarse con Spring Boot.

9. Ejecución del sistema

Para la ejecución simultánea en entorno local, abra dos terminales independientes. En una navegue a la carpeta backend/ (cd backend) y ejecute: mvn spring-boot:run. El servicio estará disponible en: http://localhost:8080. En la otra navegue a la carpeta frontend/ (cd frontend) y ejecute: npm run dev. La interfaz web estará disponible en: http://localhost:3000.

10. Verificación de la instalación

Realice la siguiente secuencia de pruebas rápidas para confirmar la correcta puesta en marcha: Abra el navegador e ingrese a http://localhost:3000. Debe visualizarse la pantalla principal de inicio de sesión del SGN de la Escuela Ing. E. Latzina. Intente ingresar con credenciales incorrectas (ej. test@et35.edu.ar / 123456). Debe retornar el mensaje de advertencia "Credenciales inválidas", lo que confirma que la comunicación frontend-backend está activa. Ingrese las credenciales del usuario administrador creado en el sembrado de datos. Si la cuenta tiene la bandera primerIngreso = true, el sistema redirigirá automáticamente a /cambiar-password.

11. Inicialización de datos (Sembrado / Seed)

Al iniciar el backend por primera vez, las clases de configuración de inicialización (ej. SeedConfig o ejecutor de comandos al inicio) evalúan la presencia del usuario administrador. Si la tabla de usuarios está vacía, el sistema genera automáticamente el perfil de administración inicial. Se debe ingresar con esta cuenta semilla en el primer arranque y completar el formulario de cambio obligatorio de contraseña antes de desplegar la plataforma a usuarios finales.

12. Configuración para desarrollo

Durante la fase de desarrollo o mantenimiento técnico, npm run dev recarga automáticamente los componentes ante cambios en el código TypeScript/React. En Spring Boot puede utilizarse el módulo spring-boot-devtools para reinicios rápidos al compilar cambios en archivos Java. Presione Ctrl + C en cada una de las terminales de comandos abiertas.

13. Configuración para un entorno de producción

Para un despliegue productivo en un servidor institucional de la escuela genere el archivo ejecutable optimizado mediante mvn clean package -DskipTests y ejecute con java -jar target/sgn-backend.jar. Genere el build de Next.js optimizado con npm run build y ejecute con npm start.

14. Copias de seguridad de PostgreSQL

Para realizar una copia de seguridad manual de la base de datos sgn_db: pg_dump -U sgn_user -h localhost sgn_db > sgn_db_backup.sql. Para importar / restaurar base de datos: psql -U sgn_user -h localhost -d sgn_db < sgn_db_backup.sql

15. Solución de problema frecuente

El Backend falla al iniciar con error de conexión JDBC. El servicio de PostgreSQL no está ejecutándose o la contraseña/usuario en application.yml son incorrectos. Verifique el estado del servicio de PostgreSQL en las herramientas administrativas de Windows

(services.msc) o con systemctl status postgresql en Linux. Pruebe la conexión con psql -U sgn_user -d sgn_db.

16. Detención del sistema

- En la consola del Frontend, presione Ctrl + C y confirme.
- En la consola del Backend, presione Ctrl + C.
- (Opcional) Detenga el servicio de PostgreSQL desde la consola de administración de servicios.

17. Estructura de carpetas representativa

```
sgn-notas/
├── backend/
│   ├── src/
│   │   └── main/
│   │       ├── java/com/sgn/
│   │       │   ├── config/       # Configuración de Seguridad, JWT y CORS
│   │       │   ├── ctrl/         # Controladores REST
│   │       │   ├── dto/          # Objetos de Transferencia de Datos
│   │       │   ├── ent/          # Entidades JPA (Usuario, Materia, etc.)
│   │       │   ├── repo/         # Repositorios Spring Data JPA
│   │       │   └── srv/          # Lógica de Negocio y Servicio de Mail
│   │       └── resources/
│   │           └── application.yml
│   └── pom.xml
└── frontend/
    ├── app/                  # Rutas de Next.js (App Router)
    ├── components/           # Componentes de Interfaz de Usuario
    ├── lib/                  # Cliente de API e integración de peticiones
    ├── .env.local
    └── package.json
```